

Anatya.ai Knowledge Hub – Production Implementation Blueprint

Goal

Build a production-ready AI Knowledge Assistant inside the Anatya panel where:

Admin: - Upload PDFs - Add URLs - Add YouTube videos - Delete/update sources

System: - Automatically parses content - Creates embeddings - Stores vectors - Retrieves answers with citations

User: - Asks questions - Receives structured answers - Sees source citations - Gets confidence score

FINAL TECH STACK (LOCKED)

Frontend: Next.js Backend: FastAPI Embeddings: BGE-small-en (local) Vector DB: ChromaDB (persistent)
Metadata DB: PostgreSQL Chat Memory: Redis LLM: Ollama Llama 3.1 (local) PDF Parser: Unstructured URL
Scraper: Firecrawl Video Parser: YouTube Transcript API Deployment: Docker VPS

SYSTEM ARCHITECTURE OVERVIEW

User Query → Retrieval Engine → Hybrid Search (Vector + Keyword) → Context Builder → LLM Answer
Generator → Citation Engine → Confidence Engine → Panel Response

Admin Upload → Ingestion Pipeline → Chunking → Embedding → Vector Storage → Metadata Storage

PHASE 1 – SERVER SETUP

Step 1: VPS Requirements

Minimum: - 4 CPU - 8GB RAM - 80GB SSD

Recommended: - 8 CPU - 16GB RAM

Install:

PHASE 2 – PROJECT STRUCTURE

knowledge-hub/

backend/ frontend/ chromadb/ assets/ pdfs/ videos/ urls/ models/ docker/

PHASE 3 – OLLAMA MODEL SETUP

Install Ollama

Pull model:

```
ollama pull llama3.1
```

Run test:

```
ollama run llama3.1
```

Verify local API:

```
http://localhost:11434
```

PHASE 4 – EMBEDDING PIPELINE SETUP

Install dependencies

```
pip install sentence-transformers chromadb
```

Load embedding model

```
BAAI/bge-small-en
```

Chunk strategy:

```
chunk_size = 800 chunk_overlap = 120
```

Store embeddings inside persistent ChromaDB directory

PHASE 5 – CHROMADB PERSISTENT CONFIGURATION

Create persistent directory

/chromadb

Initialize collection:

knowledge_docs

Metadata fields:

source_name source_type upload_date chunk_id

PHASE 6 – POSTGRESQL METADATA DESIGN

Create tables:

sources chat_history query_logs

sources table fields:

id name type location upload_date status

chat_history fields:

user_id message response timestamp

query_logs fields:

query confidence latency created_at

PHASE 7 – REDIS MEMORY LAYER

Purpose:

Maintain conversation continuity

Example:

User: pricing batao User: India ka?

Redis enables contextual awareness

PHASE 8 – PDF INGESTION PIPELINE

Install:

`pip install unstructured`

Pipeline:

Upload PDF Extract structured text Split into chunks Generate embeddings Store in ChromaDB Save metadata in PostgreSQL

PHASE 9 – URL INGESTION PIPELINE

Install Firecrawl

Process:

Input URL Convert webpage to markdown Chunk text Generate embeddings Store vectors Save metadata

PHASE 10 – YOUTUBE INGESTION PIPELINE

Use transcript API

Pipeline:

Extract transcript Split every 30 seconds Generate embeddings Store vectors Attach timestamps

PHASE 11 – HYBRID SEARCH ENGINE

Implement:

Vector Search + BM25 Keyword Search

Retrieve:

Top 20 results

Then:

Rerank top 5

Send to LLM

PHASE 12 – CONTEXT BUILDER ENGINE

Merge retrieved chunks

Remove duplicates

Sort by:

freshness similarity source_priority

Create final prompt context

PHASE 13 – LLM ANSWER GENERATION ENGINE

Send prompt to:

Ollama Llama 3.1

Response format:

Answer Steps Example Sources Confidence Score

PHASE 14 – CONFIDENCE SCORING ENGINE

Combine:

vector similarity source agreement recency score

Output:

confidence = 0.0 – 1.0

If < 0.6

Trigger support suggestion message

PHASE 15 – CITATION ENGINE

Attach:

PDF name URL source Video timestamp

Frontend sidebar opens source reference

PHASE 16 – ADMIN PANEL FEATURES

Upload PDF Add URL Add Video Delete Source Re-index Source

Display:

Total documents Total chunks Failed ingestion Recent uploads

PHASE 17 – USER PANEL CHATBOT FEATURES

Ask question Streaming answer Citation sidebar Confidence score Conversation memory

PHASE 18 – FASTAPI ENDPOINT DESIGN

Endpoints:

POST /ask POST /upload/pdf POST /upload/url POST /upload/video DELETE /source GET /sources GET /logs

PHASE 19 – NEXT.JS CHAT UI

Components:

Chat window Citation sidebar Confidence badge Streaming response renderer

PHASE 20 – DOCKER DEPLOYMENT STRUCTURE

Services:

backend frontend postgres redis chromadb ollama

Command:

docker compose up -d

PHASE 21 – AUTO REINDEX SCHEDULER

Daily cron job:

Check updated URLs Re-embed changed content Refresh vectors

PHASE 22 – PRODUCTION MONITORING

Track:

query latency confidence drops missing knowledge areas popular queries

Store inside PostgreSQL

PHASE 23 – PERFORMANCE OPTIMIZATION

Add later:

BGE reranker query caching chunk deduplication parallel embedding

PHASE 24 – SECURITY BEST PRACTICES

Restrict admin routes Validate uploads Limit file size Sanitize URLs Use JWT authentication

PHASE 25 – FINAL GO-LIVE CHECKLIST

Server running Docker stable Ollama active Embeddings working ChromaDB persistent Redis connected PostgreSQL connected Admin upload working Chatbot answering Citations visible Confidence scoring active Logs recording

SYSTEM READY FOR PANEL INTEGRATION

RESULT

After completing this blueprint:

You will have a production-grade AI Knowledge Assistant integrated into the Anatya panel capable of answering documentation-based questions with citations and confidence scoring using fully self-hosted infrastructure.